MATH DIRECTED READING PROGRAM / MEETING 5

Self-Attention as a Matrix Operator

Self-contained guided reading and problem sheet | Revision 3.0

Aim. Compute attention for all positions at once, study what happens when positions are reordered, restrict information flow with a mask, and assemble several attention heads.

Background and pacing. Matrix products, permutation matrices, and the softmax formula reviewed below. Problems 5.1–5.3 introduce the operator; 5.4 and 5.5 connect it to multiple heads and next-token targets. The optional rank question is not required to continue.

Using the self-checks. After each numbered problem, try its short self-checks with the worked solution covered. Definitions may stay open. Give a reason, not only a yes/no answer; these checks guide discussion rather than add a timed test.

5.1 Stack token representations into a matrix

CONVENTION BRIDGE: COLUMN VECTORS, STORED AS ROWS

A **token** is a discrete symbol processed by the model. A **sequence** of tokens has ordered positions $1, \dots, n$. Let its token representations be columns $x_i \in \mathbb{R}^{d_{\text{model}}}$. The **model dimension** d_{model} is the number of features, or coordinates, carried at each position. Stack their transposes:

$$X = \begin{pmatrix} x_1^\top \\ \vdots \\ x_n^\top \end{pmatrix} \in \mathbb{R}^{n \times d_{\text{model}}}.$$

Rows index positions and columns index features. This is the row-stacked convention used in the Transformer matrix formulas. It does not change an individual vector's mathematical meaning.

If $q_i = W_Q^{\text{col}} x_i$ in Meeting 4, then $q_i^\top = x_i^\top (W_Q^{\text{col}})^\top$. Define $W_Q^{\text{row}} = (W_Q^{\text{col}})^\top$, and similarly for K, V. From now through Meeting 6, the symbols W_Q, W_K, W_V refer to these *row-oriented* matrices. No silent transpose is intended.

Choose shared trainable matrices

$$W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \qquad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v},$$

and compute

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

These are **self-attention** projections because all three arrays are formed from the same sequence X. Their ith rows are $q_i^\top, k_i^\top, v_i^\top$, respectively. The subscript in d_k denotes key/query dimension; K itself is the key matrix.

Worked shape check. With $n = 3$, $d_{\text{model}} = 4$, $d_k = 2$, and $d_v = 3$, the arrays Q, K, V have shapes 3×2, 3×2, and 3×3. Thus $QK^\top$ has shape 3×3, one entry for every query-position/key-position pair.

Convention in code. PyTorch's `Linear` stores its parameter matrix as output-by-input and applies its transpose to row-stacked inputs. Thus its stored matrix corresponds to $W_Q^\top$ in the row-oriented formula above. Both conventions are standard; distinguish a mathematical map from how its coefficients are stored.

5.1 / FROM MANY DOT PRODUCTS TO ONE FORMULA

SETUP: SOFTMAX ACTS ACROSS EACH ROW

Define the score matrix $S = QK^\top/\sqrt{d_k} \in \mathbb{R}^{n\times n}$. Its entry is $S_{ij} = q_i^\top k_j/\sqrt{d_k}$. The **row-wise softmax** is

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{l=1}^{n} e^{S_{il}}}.$$

The denominator holds query position i fixed and sums over key positions. The resulting A is the **attention matrix**. The output is

$$O = AV \in \mathbb{R}^{n\times d_v}, \qquad o_i = \sum_j A_{ij} v_j.$$

A matrix is **row-stochastic** if its entries are nonnegative and every row sums to one. That says nothing, by itself, about its column sums. In this problem there is no mask and no added position information.

Worked row. If one row of scores is $(\log 2, 0)$, that row of A is $(2/3, 1/3)$. Multiplying by V returns $\frac{2}{3}v_1 + \frac{1}{3}v_2$ at the corresponding query position.

Guided questions

(a) Check every shape in $X \to (Q, K, V) \to S \to A \to O$. Verify the coordinate formula for O by matrix multiplication.

(b) Take $n = d_{\text{model}} = d_v = 2$, $d_k = 1$, with

$$X = I_2, \qquad W_Q = W_K = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad W_V = I_2.$$

Compute Q, K, V, S, A, O. Leave $e = \exp(1)$ unevaluated. In particular, check that the second attention row is uniform while the first is not.

(c) Prove that A is row-stochastic. Use part (b) to test whether its columns sum to one. Is the attention output, even when $d_v = d_{\text{model}}$, necessarily the original input?

(d) Explain why O need not be a probability vector over a vocabulary. Which index set is normalized by A, and what do the columns of V represent?

OPTIONAL SHAPE TRANSLATION: CROSS-ATTENTION

Queries can instead come from n_q positions while keys and values come from n_k source positions. With $Q \in \mathbb{R}^{n_q\times d_k}$, $K \in \mathbb{R}^{n_k\times d_k}$, and $V \in \mathbb{R}^{n_k\times d_v}$, check that the same formula gives $A \in \mathbb{R}^{n_q\times n_k}$ and $O \in \mathbb{R}^{n_q\times d_v}$. It need not produce a square attention matrix.

Translate back into machine-learning language. Matrix self-attention is the single-query weighted average calculated once for each query position, using shared projection matrices.

SELF-CHECK 5.1

1. For cross-attention with 2 query positions, 5 key/value positions, $d_k = 3$, and $d_v = 4$, give the shapes of the score, attention, and output arrays. Over how many entries does each softmax normalize?
2. What does entry A_{ij} mean? Why does A being row-stochastic say nothing about whether feature coordinates of O sum to one?

5.2 Reordering positions: equivariance, not invariance

REVIEW: WHAT A PERMUTATION MATRIX DOES

A permutation matrix $P \in \mathbb{R}^{n\times n}$ has exactly one 1 in each row and column and zeros elsewhere. Left multiplication PX reorders the rows of X, and $P^{-1} = P^\top$. For a square score matrix, $PSP^\top$ reorders both query and key positions by the same permutation.

A sequence-to-sequence map F is **permutation equivariant** if $F(PX) = PF(X)$: reordering the inputs reorders the outputs correspondingly. It is **permutation invariant** if $F(PX) = F(X)$: the output itself is unchanged. These are different claims.

Let $F(X) = A(X)V(X)$ be the unmasked self-attention operator of 5.1, with fixed shared matrices and no positional information.

Worked reordering. For $n = 2$, $P = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ exchanges the two positions. If X has rows $x_1^\top, x_2^\top$, then PX has rows $x_2^\top, x_1^\top$. Reordering features within each row would be a different operation and is not what P does here.

Guided questions

(a) Show $Q(PX) = PQ(X)$, and likewise for K, V. Use these identities to derive $S(PX) = PS(X)P^\top$.

(b) First show softmax of a permuted vector is the corresponding permutation of its softmax, by reindexing the denominator. Then apply this row by row to prove

$$\mathrm{softmax}_{\mathrm{row}}(PSP^\top) = P\,\mathrm{softmax}_{\mathrm{row}}(S)P^\top.$$

(c) Complete the calculation

$$F(PX) = (PA(X)P^\top)(PV(X)) = PF(X).$$

Identify where $P^\top P = I$ is used.

(d) Use the example in 5.1(b) to show that generally $F(PX) \neq F(X)$. Why does that not contradict the equivariance you just proved?

(e) An average over output positions is $G(X) = n^{-1}\sum_i F(X)_{i,:}$. Use $\mathbf{1}^\top P = \mathbf{1}^\top$ to show that this particular *pooled* output is invariant. Pooling has changed the type of output.

SCOPE OF THE SYMMETRY STATEMENT

This proof assumes no positional vectors, no position-dependent score terms, and no fixed order-dependent mask. It shows that content-based, unmasked self-attention alone has no distinguished ordering of positions. A causal mask already encodes an ordering; do not apply this theorem unchanged to the next problem.

Translate back into machine-learning language. Translate "permutation equivariant" into a concrete statement about swapping two input rows and comparing the corresponding output rows.

SELF-CHECK 5.2

1. Swapping two rows swaps the two corresponding output rows but changes the output array. Is this evidence for equivariance, invariance, or neither? Explain the equality being tested.
2. If row 1 and row 2 use different projection matrices, which identity at the start of the equivariance proof may fail? Identify the role of shared parameters.

5.3 A causal mask restricts information flow

SETUP: ALLOW ONLY THE CURRENT POSITION AND ITS PREFIX

For next-token prediction at position i, we want access to the input tokens through i, but not later ones. A **causal attention mask** specifies the allowed set $\{j : j \le i\}$. Formally set

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i, \end{cases} \qquad A = \mathrm{softmax}_{\mathrm{row}}(S + M).$$

The symbol $-\infty$ is shorthand for omitting forbidden terms, with $e^{-\infty} = 0$. Equivalently, for finite scores,

$$A_{ij} = \begin{cases} e^{S_{ij}} / \sum_{l \le i} e^{S_{il}}, & j \le i, \\ 0, & j > i. \end{cases}$$

This is inclusive of the diagonal: position i may use its own input representation when predicting token $i + 1$. Every row has at least one allowed key.

Worked row. If the three scores in row 2 are all zero, masking changes its weights from $(1/3, 1/3, 1/3)$ to $(1/2, 1/2, 0)$. It does not leave the allowed weights unnormalized.

Guided questions

(a) Write M for $n = 3$. If all scores are zero and scalar values are $(2, 4, 8)$, compute all three output values. Which inputs influence each output?

(b) Why does replacing forbidden scores by 0 fail to exclude them? Explain the role of exponentiation before normalization.

(c) Suppose an arbitrary mask excludes every key in some row. What happens to the denominator in the allowed-set definition? Why must a mathematical specification or implementation handle this case explicitly?

(d) For $n = 2$, zero scores, scalar values $(2, 8)$, and the inclusive causal mask, compute the two outputs. Swap the values while holding the mask fixed and recompute. Is the new result the swap of the old result?

FIXING A MASK IS NOT THE SAME AS RELABELING IT

If positions and their allowed connections are *both* relabeled, the mask changes to $PMP^{\top}$, and the earlier reindexing calculation applies to the masked operator. An ordinary left-to-right causal mask is instead fixed to the chosen order, so arbitrary permutations need not be symmetries.

Translate back into machine-learning language. "Causal" here is a precise restriction on which input positions may contribute, not a claim of causal inference. A mask changes information access; a positional encoding is a different way to supply position information.

SELF-CHECK 5.3

1. For three zero scores and allowed keys $\{1, 3\}$, write the normalized weight row. Would applying softmax first and then merely zeroing entry 2 give the same row?
2. Two sequences agree in their first two input representations but differ afterward. Can inclusive causal self-attention change its second output row? State which inputs its query, allowed keys, and values use.

5.4 Several heads, then one output projection

SETUP: INDEPENDENTLY PARAMETERIZED ATTENTION OPERATIONS

An **attention head** is one attention computation with its own projection matrices. For n_{heads} heads indexed by a, let

$$Q_a = XW_Q^{(a)}, \quad K_a = XW_K^{(a)}, \quad V_a = XW_V^{(a)},$$

where $W_Q^{(a)}, W_K^{(a)} \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_V^{(a)} \in \mathbb{R}^{d_{\text{model}} \times d_v}$. Use the same permitted-position mask M in all heads, and set

$$A_a = \text{softmax}_{\text{row}}(Q_a K_a^\top / \sqrt{d_k} + M), \qquad O_a = A_a V_a \in \mathbb{R}^{n \times d_v}.$$

For unmasked attention, set every entry of M to 0.

Concatenation joins the head outputs along the feature dimension, not the position dimension:

$$C = [\, O_1 \mid O_2 \mid \cdots \mid O_{n_{\text{heads}}} \,] \in \mathbb{R}^{n \times (n_{\text{heads}} d_v)}.$$

A learned **output projection** $W_O \in \mathbb{R}^{(n_{\text{heads}} d_v) \times d_{\text{model}}}$ then gives **multi-head attention (MHA)**:

$$\text{MHA}(X) = CW_O \in \mathbb{R}^{n \times d_{\text{model}}}.$$

All attention projections on this sheet are bias-free. "Independent heads" means separately parameterized, not a claim that their outputs are statistically independent.

Worked concatenation. If two heads return row vectors $(1, 2)$ and $(3, 4)$ at a certain position, concatenation produces $(1, 2, 3, 4)$. It neither adds them to $(4, 6)$ nor creates a second position. Multiplication by W_O then mixes these head coordinates within that position.

Guided questions: read the architecture

(a) Take $n = 3$, $d_{\text{model}} = 4$, $n_{\text{heads}} = 2$, and $d_k = d_v = 2$. List the shapes of every parameter matrix, every A_a, O_a, the concatenation C, and the final output.

(b) Let $W_O = \begin{pmatrix} I_2 \\ I_2 \end{pmatrix}$ in a two-head toy case with $d_v = d_{\text{model}} = 2$. What does this particular output projection do to the two head vectors? How does that differ from concatenation alone?

Translate back into machine-learning language. Each head can compute its own distribution over source positions. The output projection combines the resulting feature vectors; it is not the vocabulary classifier's W_{out}.

5.4 / COUNT PARAMETERS WITHOUT COUNTING ACTIVATIONS

REVIEW: WHAT IS SHARED, AND WHAT IS RECOMPUTED?

A **parameter count** counts trainable scalar entries in stored arrays. Each head uses the same matrices at every position. By contrast, the entries of Q_a, K_a, V_a, A_a, O_a are **activations**: intermediate values computed in the forward pass. They are not new trainable parameters for each example.

A common dimension choice is

$$d_k = d_v = d_{\text{model}}/n_{\text{heads}},$$

assuming divisibility. This keeps the *total* head-output width equal to the model width. It is a design choice, not a requirement of the definition of attention.

Guided questions: count and interpret

(c) Count the entries in $W_Q^{(a)}, W_K^{(a)}, W_V^{(a)}$ for one head, then in all heads and W_O. Obtain

$$n_{\text{heads}} d_{\text{model}}(2d_k + d_v) + (n_{\text{heads}} d_v) d_{\text{model}}.$$

Simplify under the common dimension choice to $4d_{\text{model}}^2$.

(d) For the dimensions in 5.4(a), compute the count numerically. Does changing the sequence length n change it? Separately count the entries across all attention matrices A_a.

(e) At fixed d_{model}, increasing n_{heads} under the common dimension rule changes two things: the number of attention matrices and the feature dimension within each head. Identify both changes. Does "more heads" mean "strictly more parameters" in this bias-free calculation?

(f) If all heads had identical projection matrices and identical widths, what would they compute on the same input? Explain why separate parameters permit different patterns but do not force distinct useful patterns to be learned.

A CONCRETE TWO-HEAD EXAMPLE: DIFFERENT WEIGHTED SUMS

Use $X = I_2$, two heads, $d_k = d_v = 1$, and no mask. Choose

$$W_Q^{(1)} = \begin{pmatrix} \log 3 \\ 0 \end{pmatrix}, \quad W_Q^{(2)} = \begin{pmatrix} -\log 3 \\ 0 \end{pmatrix}, \quad W_K^{(1)} = W_K^{(2)} = W_V^{(1)} = W_V^{(2)} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Thus both heads have values $(1, 0)^\top$, but their first query has opposite sign. Keep their scalar outputs separate before applying $W_O = \begin{pmatrix} 1 & 0 \\ -1 & 0 \end{pmatrix}$.

(g) Compute both first-row score vectors, both attention-weight vectors, and both scalar head outputs. Concatenate those outputs and multiply by W_O. What information would be discarded if the two scalar head outputs were averaged before applying this projection?

(h) At position 2, both queries are zero. Compute that output row. Explain why separately parameterized heads *can* produce different summaries without guaranteeing they differ at every input or every position.

Translate back into machine-learning language. Distinguish parameter count, intermediate-array size, and number of attention patterns. They measure different aspects of an architecture.

SELF-CHECK 5.4

1. In the concrete two-head example, the first-position outputs are concatenated before W_O. Does replacing their pair by its average preserve enough information to recover their difference for arbitrary head outputs?
2. With fixed model width and $d_k = d_v = d_{\text{model}}/n_{\text{heads}}$, does doubling the number of heads necessarily double projection parameters? Which intermediate arrays increase in number?

5.5 Align the causal mask with next-token targets

SETUP: INPUTS AND LABELS OCCUPY DIFFERENT ROLES

A token is a discrete symbol with an ID in a fixed vocabulary of C types. Take four observed token IDs (t_1, t_2, t_3, t_4). For this training example, input rows represent (t_1, t_2, t_3) and the three target labels are (t_2, t_3, t_4). A **prefix** is the initial part of a sequence. Assume row j initially depends only on token t_j, then use inclusive causal attention and only position-wise operations afterward.

Let h_i be the final column representation at position i. An output head computes $z_i = W_{\text{out}} h_i + b \in \mathbb{R}^C$ and $p_i = \text{softmax}(z_i)$. Its predicted distribution is for the *next* token, not its own input token.

Output position	Allowed input prefix	Target label
1	(t_1)	t_2
2	(t_1, t_2)	t_3
3	(t_1, t_2, t_3)	t_4

Worked alignment. Row 2 may read its own input token t_2. This is not the answer it must predict: its label is t_3. Row 3 contains t_3 as input, so row 2 must not read row 3. Targets enter the loss calculation, not the allowed forward inputs.

PROBABILITY REVIEW: FACTOR BY THE CHAIN RULE

For events with the needed conditional probabilities defined, $\Pr(A, B) = \Pr(A)\Pr(B \mid A)$. Repeating this rule gives

$$\Pr_\theta(t_2, t_3, t_4 \mid t_1) = \prod_{i=1}^{3} \Pr_\theta(t_{i+1} \mid t_1, \ldots, t_i).$$

This factorization does *not* assume independent tokens. A model predicting each next token from earlier ones is called **autoregressive**. Writing $p_{i,j}$ for the probability assigned to vocabulary ID j, the loss for this example is the sum $L = -\sum_{i=1}^{3} \log p_{i,t_{i+1}}$.

Guided questions

(a) Explain why including the diagonal in the causal mask does not reveal the target at the same output position. Which forbidden row would reveal row 1's target directly?

(b) Write the three one-hot targets and expand the three cross-entropy terms. Taking $-\log$ of the displayed conditional probability product gives which expression? Where would averaging introduce a factor?

(c) The full input array is available during training. Explain why the mathematical dependencies of p_1 and p_2 nevertheless exclude later input rows. Separate availability in storage from use by the computation.

Translate back into machine-learning language. A triangular mask answers "which representations may this position use?" Target alignment answers "which token is this position asked to predict?" Both are needed to state the training task without future-token leakage.

SELF-CHECK 5.5

1. Keep inputs (t_1, t_2, t_3) fixed and change only the last target t_4. Before any optimizer step, which probability vectors stay unchanged and which loss term may change?
2. At output position 2, is reading input token t_2 a leak of its target? Is reading row 3 a leak? Explain using the target alignment, not merely the word causal.

Optional extension 5.E: rank after softmax

5.E / A LOW-RANK SCORE MATRIX NEED NOT GIVE LOW-RANK ATTENTION

The score matrix in 5.1(b) is $S = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$, of rank one. Compute the determinant of

$$\mathrm{softmax}_{\mathrm{row}}(S) = \begin{pmatrix} e/(e+1) & 1/(e+1) \\ 1/2 & 1/2 \end{pmatrix}.$$

Does the attention matrix still have rank one? Matrix multiplication obeys rank bounds, but row-wise softmax is not matrix multiplication by a fixed linear map. A rank bound on $QK^\top$ does not pass unchanged through this nonlinear operation.

SELF-CHECK 5.E

1. Does a low-rank factorization of $QK^\top$ automatically prove that $\mathrm{softmax}_{\mathrm{row}}(QK^\top)$ has the same rank bound? Identify the nonlinear step that invalidates this inference.

Discussion checkpoint and selected hints

READY TO MOVE ON WHEN...

Compute one masked attention row with the correct normalization, check all matrix shapes, and explain the difference between equivariance and invariance. Use the two-head example to distinguish concatenation from averaging, then align a position with its next-token target in 5.5.
You may consult definitions and notation. Explain the choices without following a worked solution line by line. If a self-check exposes a gap, return to that problem's setup; optional extensions do not block progress.

CHECK YOUR TRANSLATIONS

A row of X represents one position; a column represents one feature. A row of A gives one query's normalized coefficients over key positions.
Permutation equivariance means corresponding reordering of outputs, not identical outputs. The theorem here is for unmasked, position-free self-attention. A fixed causal mask changes its symmetry.
Concatenation joins feature coordinates. W_O returns these coordinates to model width; W_{out} in a vocabulary head maps a final representation to class logits. The two output projections have different roles and shapes.

Suggested discussion. Present the complete numerical example in 5.1. Then explain one line of the permutation proof and one counterexample involving a fixed causal mask.

5.1(b). The score rows are $(1, 0)$ and $(0, 0)$, and $V = I_2$.

5.2(b). A permutation changes the order of the same denominator summands. It neither inserts nor removes a term.

5.3(d). With two zero-score positions, the first output is its own value and the second is the average of both values.

5.4(c). With $d_k = d_v = d_{\text{model}}/n_{\text{heads}}$, the three groups of input projections total $3d_{\text{model}}^2$, while W_O contributes d_{model}^2.

Optional rank question. The determinant is $(e-1)/(2(e+1))$, which is nonzero.

Optional reading: reconnect with standard sources

[1] Vaswani et al., *Attention Is All You Need* (2017). §§3.2.1–3.2.3: scaled dot-product attention, multi-head attention, and the uses of attention and masking. Compare the matrix shapes and the concatenation operation; the full encoder–decoder architecture is not yet required.

[2] Zhang, Lipton, Li, and Smola, *Dive into Deep Learning*. §11.5, Multi-Head Attention; §11.6, Self-Attention and Positional Encoding, focusing on the self-attention discussion. Position encodings themselves belong to a later module. For 5.5, see §9.3.1, Learning a Language Model: the autoregressive probability factorization.

[3] Prince, *Understanding Deep Learning* (2023). §§12.2–12.3: dot-product self-attention and its extensions. Track vector orientation rather than assuming every source stores token representations along the same axis.

[4] PyTorch documentation. `Linear`, formula and weight shape: the storage convention discussed in 5.1. The symmetry and rank calculations in this sheet are direct exercises, not claims about every implementation or architecture.

Readings supply context, not missing definitions. Exercises and numerical examples are original to this DRP. Prince locators refer to the 2023 first-edition organization; D2L locators refer to its 1.0.3 online edition. Use section titles when editions differ.